

Lab - 01

Fundamentals of Python Programming & Control Structures

Introduction

Python is a high-level, interpreted programming language celebrated for its elegant syntax, remarkable readability, and multi-paradigm flexibility. It has established itself as the dominant language across software development, data science, artificial intelligence, machine learning, web applications, and DevOps automation. Prior to implementing complex AI pipelines, establishing a robust coding environment is crucial. This experiment introduces the foundational syntax and control structures required to implement efficient logic in Python.

Theory

- **Python Installation:** The installation process configures the Python core interpreter on the host machine. Once added to the system PATH, Python scripts can be evaluated directly from command-line terminals or integrated development environments.
- **Coding Environment Setup:** Integrated Development Environments (IDEs) like Visual Studio Code (VS Code) and PyCharm empower engineers with syntax highlighting, IntelliSense auto-completion, isolated virtual environments, and source-level debugging pipelines.
- **Python Output & Input:** The `print()` function transmits text or object representations to the standard output stream. Conversely, the `input()` function halts program execution to receive user data from the standard input stream, storing it implicitly as a string type.
- **Conditional Statements:** Conditional execution utilizes `if`, `elif`, and `else` statements to branch code pathways dynamically based on Boolean evaluation metrics.
- **Loop Control (Break & Continue):** The `break` statement exits the nearest enclosing loop immediately, while the `continue` statement skips the remainder of the current iteration, passing execution control back to the loop header.
- **Lists:** An ordered, mutable, and heterogeneous collection type that enables data structure instantiation, slicing, indexing, and linear traversals.

□ **Iterative Loops (For, While, and Nested):** For loops are sequence-driven iterators, whereas while loops execute indefinitely as long as a baseline logical premise remains True. Nested iterations evaluate multi-dimensional coordinate spaces or structural grids.

Equipment/Software

- **Operating System:** Linux (Ubuntu Core Environment)
- **Core Runtime:** Python 3.x Interpreter
- **IDE Platform:** Visual Studio Code / PyCharm
- **Interface:** Linux Bash Terminal Program Implementations & Outputs

1. Hello World Output Function

```
msg = "Hello, World!"  
print(msg)
```

Output:

```
Hello, World!
```

2. User Input Tracking

```
name = "Amit Sarkar"  
print(f"Hello, {name}!")
```

Output:

```
Hello, Amit Sarkar!
```

3. Grade Assessment Logic

```
marks = 85  
  
if marks >= 80:  
    print("Grade: A")  
else:  
    print("Grade: B")
```

Output:

```
Grade: A
```

4. Break and Continue Simulations

```
# Break demonstration
for i in range(1, 11):
    if i == 5: break
    print(i)
```

```
# Continue demonstration
for i in range(1, 6):
    if i == 3: continue
    print(i)
```

Output for Break: 1, 2, 3, 4

Output for Continue: 1, 2, 4, 5

5. List Attributes and Nested Iterations

```
items = [10, 20, 30, 40]
print(f"First: {items[0]}, Last: {items[-1]}, Length:
{len(items)}")
```

```
# Matrix Grid Output Pattern
for r in range(3):
    print("***")
```

Output:

First Element: 10

Last Element: 40

Length of List: 4

Conclusion

This initial laboratory successfully established operational proficiency with Python. By building basic command-line logic, parsing inputs, managing control flow via conditionals, and loops, a baseline foundation was created to handle complex algorithmic workflows in subsequent experiments.

Lab - 02

Advanced Functional Paradigms & Numerical Computing with NumPy

Introduction

Modern data science and artificial intelligence rely on modular software architectures and high-performance mathematical operations. This lab covers advanced functional abstraction mechanisms—such as dynamic parameter parsing, scoping boundaries, and anonymous lambda expressions—alongside an extensive exploration of the NumPy scientific library, which forms the computational backbone for modern machine learning frameworks.

Theory

- **Functions & Return Formats:** Python functions are modular, reusable blocks instantiated via the `def` keyword. They can return multiple values packed as immutable tuples.
- **Parameter Handling:** Default parameters provide fallbacks when arguments are omitted. Dynamic parameters (`*args` for positional arguments and `**kwargs` for keyword-value pairings) support flexible, variable-length inputs.
- **Variable Scope:** Local variables exist strictly within their declaring block scope, whereas global variables persist throughout the module runtime.
- **Lambda Expressions:** Inline, anonymous functions designed for brief, functional interactions often used with `map()`, `filter()`, or `sort` routines.
- **Modules and Packages:** Modular code organization splits scripts into reusable files (.py modules) and directories (packages), aiding architectural scale.
- **NumPy Arrays & Linear Algebra:** Vectorized computing via 1D and 2D arrays minimizes loop overheads. Specialized functions like `zeros()`, `ones()`, `eye()`, `linspace()`, and `reshape()` allow easy tensor generation and structural morphing.

Equipment/Software

- **Operating System:** Linux (Ubuntu Operating System)
- **Core Runtime:** Python 3.x Language Environment

□ **Libraries:** NumPy Scientific Computing Library

□ **IDE Platform:** Visual Studio Code / PyCharm

Program Implementations & Outputs

1. Multiple Return Value Function

```
def compute(a, b):  
    return a + b, a - b  
  
s, d = compute(20, 5)  
print(f"Sum = {s}, Difference = {d}")
```

Output:

```
Sum = 25  
Difference = 15
```

2. Default Value in Functions

```
def greet(name, msg="Hello"):  
    print(f"{msg}, {name}!")  
  
greet("Amit Sarkar")
```

Output:

```
Hello, Amit Sarkar!
```

3. Dynamic Parameter Aggregation

```
def sum_all(*args):  
    return sum(args)  
  
print(f"Total = {sum_all(5, 10, 15, 20)}")
```

Output:

```
Total = 50
```

4. NumPy Vectorized Matrix Algebra

```
import numpy as np
A = np.array([1, 2, 3])
B = np.array([4, 5, 6])
print("Addition:", A + B)
print("Multiplication:", A * B)
print("Dot Product:", np.dot(A, B))
```

Output:

```
Addition = [5 7 9]
Multiplication = [4 10 18]
Dot Product = 32
```

5. Matrix Re-shaping & Matrix Generation Pipelines

```
linspace_arr = np.linspace(1, 12, 6)
matrix_2x3 = linspace_arr.reshape(2, 3)
print("Linspace:", linspace_arr)
print("Reshaped:\n", matrix_2x3)
```

Output:

```
Linspace:
[1.0 3.2 5.4 7.6 9.8 12.0]
Reshaped Matrix:
[[1.0 3.2 5.4]
 [7.6 9.8 12.0]]
```

Conclusion

This laboratory illustrated how structured code organization and efficient numeric backends improve program performance. Leveraging NumPy for vectorized computations instead of native loops drastically accelerates math workflows, laying the groundwork for matrix-heavy AI modeling.

Lab - 03

Object-Oriented Programming (OOP) Paradigms in Python

Introduction

Object-Oriented Programming (OOP) is a foundational software design paradigm that models real-world domain problems as distinct, interactable objects containing state and behavior. This experiment investigates the key structural pillars of OOP—encapsulation, inheritance, polymorphism, and abstraction—along with advanced Python constructs like magic methods and operator overloading.

Theory

- **Classes and Objects:** A class acts as a structural blueprint, while objects represent runtime instances populated with dedicated data fields.
- **Encapsulation:** Restricting direct structural modification of internal variables by routing access through clean getter and setter routines, protecting state.
- **Inheritance:** Mechanisms enabling child classes to inherit attributes and methods from base parent classes, eliminating codebase duplication.
- **Polymorphism:** Design patterns where multiple derived objects share identical method signatures but execute distinct specialized logic.
- **Abstraction:** Hiding complex low-level implementation details behind clean, minimal abstract base classes (ABCs).
- **Dunder Methods & Operator Overloading:** Predefined internal hooks (e.g., `__init__`, `__str__`, `__add__`) that customize default operator behaviors (+, -, ==) for custom domain objects.

Program Implementations & Outputs

1. Classes and Objects Demonstration

```
class Student:
    def __init__(self, name, student_id, intake):
```



```
        self.name = name

        self.student_id = student_id

        self.intake = intake

s = Student("Amit Sarkar", "22235103283", 51)

print(f"Student Name: {s.name}\nStudent ID:
{s.student_id}\nIntake: {s.intake}")
```

Output:

```
Student Name: Amit Sarkar
Student ID: 22235103283
Intake: 51
```

2. Encapsulated Banking Instance

```
class Account:

    def __init__(self, bal):

        self.__balance = bal

    def adjust(self, dep, withdr):

        self.__balance += dep - withdr

        return self.__balance

acc = Account(0)

print("Updated Balance:", acc.adjust(5000, 1500))
```

Output:

```
Updated Balance = 3500
```

3. Inheritance Setup

```
class Person:

    def __init__(self, name):

        self.name = name

class CSEStudent(Person):
```

```
def __init__(self, name, dept, student_id):
    super().__init__(name)
    self.dept = dept
    self.student_id = student_id

std = CSEStudent("Amit Sarkar", "CSE", "22235103283")
print(f"Name: {std.name}\nDepartment: {std.dept}\nID: {std.student_id}")
```

Output:

```
Name: Amit Sarkar
Department: CSE
ID: 22235103283
```

4. Polymorphic Interface Demonstrations

```
class Dog:
    def sound(self): return "Bark"

class Cat:
    def sound(self): return "Meow"

for animal in [Dog(), Cat()]:
    print(animal.__class__.__name__ + ":", animal.sound())
```

Output:

```
Dog: Bark
Cat: Meow
```

5. Operator Overloading Script

```
class Vector:
    def __init__(self, val):
        self.val = val
```

```
def __add__(self, other):  
    return self.val + other.val
```

```
A = Vector(15)  
B = Vector(25)  
print(f"A + B = {A + B}")
```

Output:

```
A + B = 40
```

Conclusion

OOP methodologies provide a structured way to manage scaling systems. Implementing encapsulation, polymorphism, and custom operators allows developers to build clean, maintainable, and highly reusable codebases for modern software engineering.

Lab - 04

Algorithmic Graph Traversals (BFS and DFS)

Introduction

Graph structures model intricate relational systems like computer networks, knowledge graphs, and decision spaces. Efficiently exploring these networks requires robust traversal algorithms. This laboratory covers the design, implementation, and evaluation of two fundamental search techniques: Breadth-First Search (BFS) and Depth-First Search (DFS).

Theory

- **Adjacency List Structure:** A clean memory representation using key-value dictionaries, where every key maps to a list of its neighboring nodes.
- **Breadth-First Search (BFS):** An iterative exploration strategy that scans nodes level-by-level from a chosen root. It uses a First-In, First-Out (FIFO) Queue to manage discovery order.
- **Depth-First Search (DFS):** A deep exploration strategy that tracks paths as far as possible before backtracking. It uses a Last-In, First-Out (LIFO) Stack, implemented either recursively or via lists.

Algorithmic Trace Pipelines

Given Graph Adjacency Mapping:

Queue Log:

- Init: [A] -> Visit A
- Step 1: [B, C] -> Visit B
- Step 2: [C, D, E] -> Visit C
- Step 3: [D, E] -> Visit D

- Step 4: [E] -> Visit E -> Done

Final BFS Route: A -> B -> C -> D -> E

DFS Traversal Order (Stack Tracker):

Stack Log:

- Init: [A] -> Visit A
- Step 1: [B, C] -> Visit C
- Step 2: [B, E] -> Visit E
- Step 3: [B] -> Visit B
- Step 4: [D] -> Visit D -> Done

Final DFS Route: A -> B -> D -> E -> C

Conclusion

This final experiment highlighted the operational mechanics of graph traversals. BFS is well-suited for shortest-path operations in unweighted graphs, while DFS is ideal for connectivity tests, topological sorting, and deep path analysis, providing core routing techniques for AI problem-solving.